

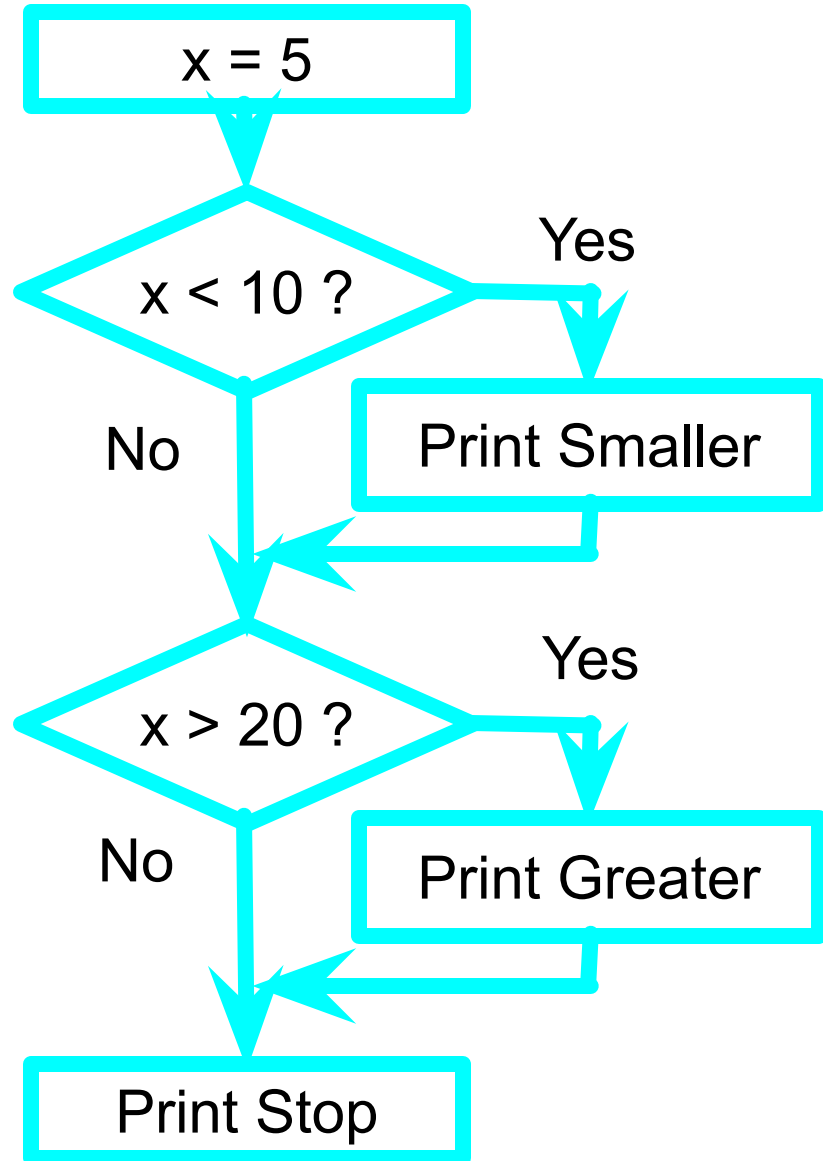
Chapter 4&6

Conditionals and Iterative (s) (loops)

Outline

- Boolean Expression and Logical Operators
- Conditional Execution
- Chained and Nested Conditions
- Keyboard input and Defensive Programming

Conditional Steps



Program:

```
x = 5
if x < 10:
    print('Smaller')
if x > 20:
    print('Greater')
print('Stop')
```

Output:

Smaller
Stop

Comparison Operators

- **Boolean expressions** ask a question and produce a Yes or No result which we use to control program flow
- **Boolean expressions** using **comparison operators** evaluate to True / False or Yes / No
- Comparison operators look at variables but do not change the variables

Python	Meaning
<	Less than
<=	Less than or Equal to
==	Equal to
>=	Greater than or Equal to
>	Greater than
!=	Not equal

Comparison Operators

```
age = 19
if age == 19 :
    print('Equals 19')
if age > 18 :
    print('Greater than 18')
if age >= 18 :
    print('Greater than or Equals 18')
if age < 20 :
    print('Less than 20')
if age <= 19 :
    print('Less than or Equals 19')
if age != 20 :
    print('Not equal 20')
```

Equals 19

Greater than 18

Greater than or Equals 18

Less than 20

Less than or Equals 19

Not equal 20

Example

```
age = 18
print("Now")
if age == 18:
    print("age is", age)
    print("age still", age)
print("After a year")
print("age changes")
age = age + 1
if age == 19:
    print("age is", age)
    print("age is still", age)
print("Go to college")
```

Output:

Now
age is 18
age still 18
After a year
age changes
age is 19
age is still 19
Go to college

Indentation

- Increase indent after an if statement or for statement (after :)
- Maintain indent to indicate the scope of the block (which lines are affected by the if/for)
- Reduce indent back to the level of the if statement or for statement to indicate the end of the block
- Blank lines are ignored - they do not affect indentation
- Comments on a line by themselves are ignored with regard to indentation

Indentationcont'd

- Indentation is crucial in Python, especially when working with conditionals, loops, and defining functions
- Python relies on indentation
 - to define blocks of code,
 - to define the scope of loops, conditionals, functions, and other constructs.

Why indentation in Python

- **Syntax Requirement**

- Python uses indentation to define the beginning and end of code blocks.

- if condition:

- # Indented block

- print("Condition is true")

- The indented lines after the 'if' are part of the 'if' block
 - Without proper indentation, Python will raise an **IndentationError**

Why indentation in Python

- **Readability**

- Proper indentation makes the code more readable and easier to understand.
- It visually separates different blocks of code, making it clear which statements are controlled by which conditions or loops.

- **Logical Grouping**

- Indentation helps group statements logically.
- Statements that should execute together under a certain condition or within a loop are grouped under the same level of indentation.

- **Avoiding Errors**

- Indentation prevents syntax errors and logical errors.
- Consistent indentation ensures that the code executes in the intended order and scope.

Example:

```
age = int(input("your age: ")) # 18 or 17 or 16
if age >= 18:
    print("You are an adult.") # This line is indented and part of the if block
print("you can go for voting.") # This line is not indented and executed
                                regardless of the if condition
```

```
age = 18
if age >= 18:
    print("You are an adult.")
    if age == 18: #nested condition
        print("You just turned 18!")
```

'''Note that the second 'if' statement is indented in the first 'if' block making it clear that it is a nested condition '''

Common errors due to incorrect indentation

```
#IndentationError
```

```
if condition:
```

```
print("This will cause an IndentationError") # This line should be indented
```

```
#Logical error
```

```
if condition:
```

```
    print("This is correct")
```

```
print("This will always execute, regardless of the condition")
```

Best practice?

- Use spaces or tabs consistently (4 spaces per indentation level)
- Use an IDE or code editor that automatically handles indentation for you.
- Avoid mixing tabs and spaces in the same project to prevent '**TabError**'

If statement

- An `if` statement is used to implement a decision. When a condition is satisfied (that is, `True`), one set of statements is executed. Otherwise, another set of statements is executed.
- The syntax of an `if` statement with No else branch is shown below. The **colon** indicates the header.

Syntax	Example	Flow chart for <code>if</code> statement with No else branch
<pre>if condition : statement(s)</pre>	<pre>age = int(input("age: ")) if age < 18 : age = age + 1 # True branch - execute the # statement above only if the # condition is True</pre>	<pre>graph TD; Entry(()) --> Decision{age > 18?}; Condition((Condition)) -.-> Decision; Decision -- True --> Process[age = age + 1]; Decision -- False --> NoElse((No else branch)); Process --> Exit(()); NoElse --> Exit;</pre>

- **Note:** Indent the block of `statement(s)` in the `True` branch.

If ... else... statement

- The syntax of an `if` statement with `else` branch is shown below.

Syntax	Example	Flow chart for <code>if</code> statement with <code>else</code> branch
<pre>if condition : statement(s) else : statement(s)</pre>	<pre>qty = int(input("quantity: ")) uprice = 10 if qty > 5 : totalP = uprice*qty - 1 # True branch - execute # only if the condition # is True else : totalP = uprice*qty # False branch - execute # only if the condition # is False</pre>	<pre>graph TD; Entry(()) --> Decision{qty > 5?}; Decision -- True --> Process1[totalP = uprice*qty - 1]; Decision -- False --> Process2[totalP = uprice*qty]; Process1 --> Exit(()); Process2 --> Exit;</pre>

- Note: Align `if` and `else` Indent the block of statement(s) in each branch.

If..elif...else Statements

- The syntax of an `elif` statement is as follows.

Syntax	Example
<pre>if condition : statement(s) elif condition : statement(s) else : statement(s)</pre>	<pre>scoreA = int(input("Enter a score for player A: ")) scoreB = int(input("Enter a score for player B: ")) if scoreA > scoreB : print("A won") elif scoreB > scoreA : print("B won") else : print("Game tied")</pre>

- **Note:** Align `if`, `elif` and `else`
- Indent the statements in each branch.

Notes on conditionals

- You have to have at least one statement inside each branch but you can use the *pass* command while stubbing

```
if x < 0:  
    pass #Handle neg values
```

- You can have as many `elif` branches as you need
 - This replaces the *select* and *switch* commands in other languages
- You can create complex conditionals
 - Conditionals can be nested
 - Use Boolean logic operators `and`, `or`, and `not` to chain together simple conditions

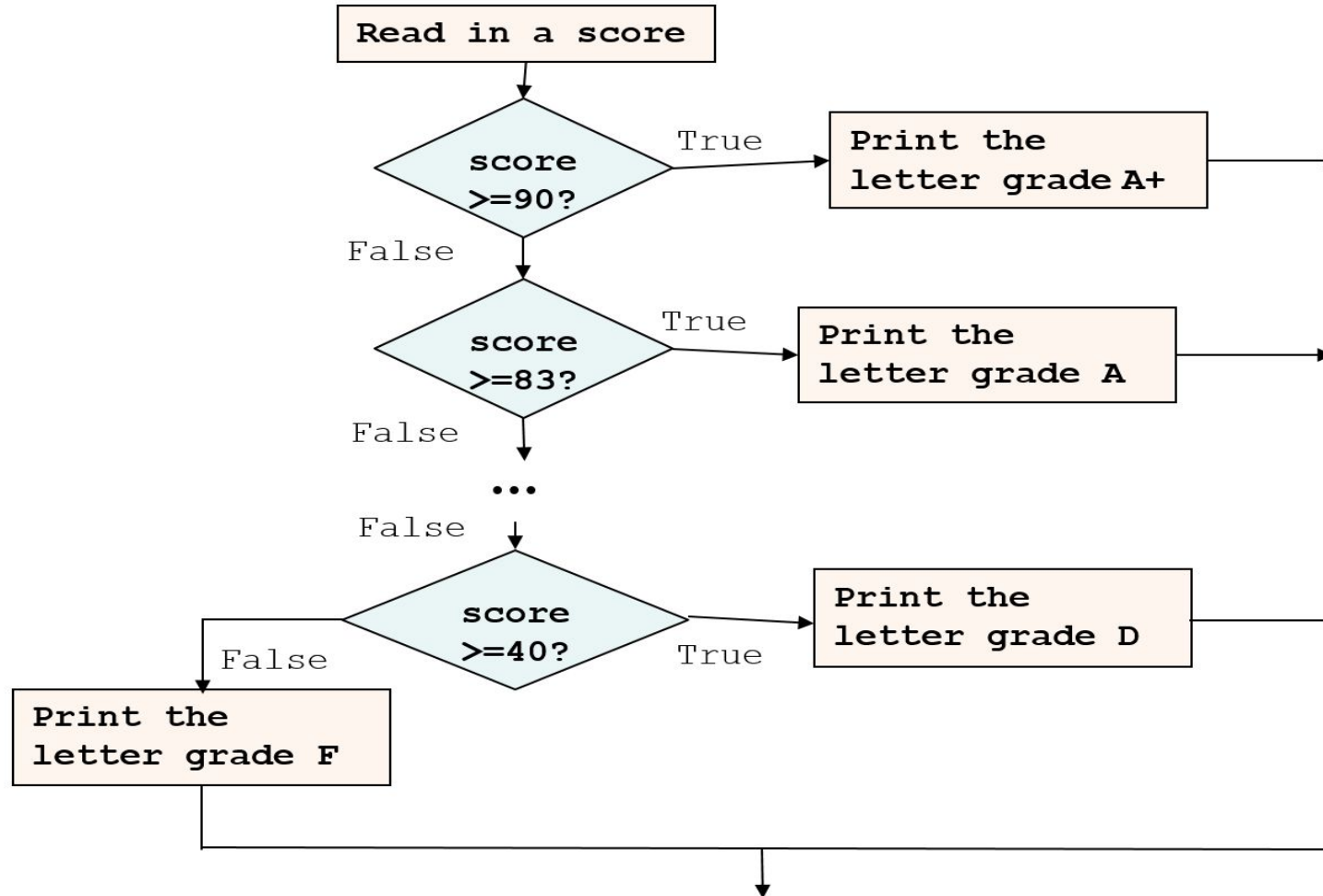
Problem Solving Exercise

• Problem statement

- Write a program that inputs an integer from the keyboard. This integer is the score of an exam. Use an appropriate prompt.
- Assign a letter grade to the score, according to the rule shown in the table.
- Include appropriate description while printing the letter grade

Score	Grade
<=100, but >=90	A+
<90, but >=83	A
<83, but >=80	A-
<80, but >=75	B+
<75, but >=68	B
<68, but >=65	B-
<65, but >=60	C+
<60, but >=50	C
<50, but >=45	C-
<45, but >=40	D
<40, but >=0	F

Problem solving – the flow chart below illustrates a possible solution.



Problem Solving Exercise cont'd

- Problem solving - Convert the pseudo code below into a sequence of Python statements in your program.

Input

Process the input and display the correct output (steps 2 to 11).

1. Read in an integer and store it in the variable `score`.*
2. Write the statements below to check if the `score` is greater than or equal to 90, and then print the letter grade `A` in the True branch.

```
if score <= 100 and score >= 90:
```

```
    print("Grade A+")           # True branch
```
3. Add the following statements to check if the `score` is less than 90 but greater than or equal to 83, and then print the letter grade `A`:

```
elif score < 90 and score >= 83:
```

```
    print("Grade A")           # True branch
```
4. Write `elif` and `print` statements similar to step 3 to check if the `score` is less than 83 but greater than or equal to 75, and then print the letter grade `B+`.

*Hint: First use the `input` function to read in a numeric value typed in at the keyboard. Then use the function `int` to convert the input string to an integer and store it in the variable `score`.

Problem Solving Exercise cont'd

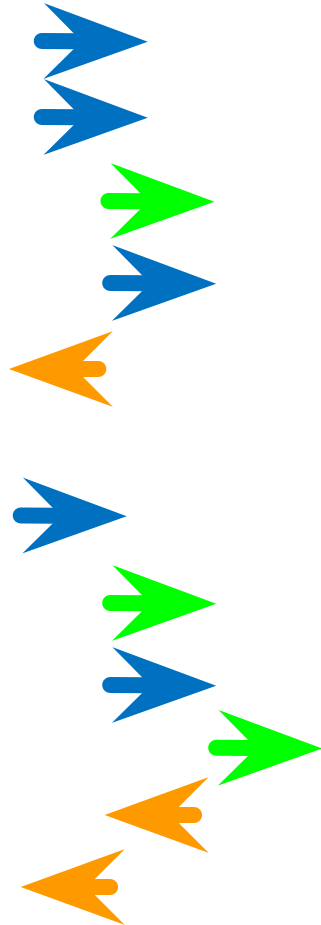
• Problem solving (continued)

5. Write `elif` and `print` statements to check if the `score` is less than 75 but greater than or equal to 68, and then print the letter grade B.
6. Write `elif` and `print` statements to check if the `score` is less than 68 but greater than or equal to 65, and then print the letter grade B-
7. Write `elif` and `print` statements to check if the `score` is less than 65 but greater than or equal to 60, and then print the letter grade C+
8. Write `elif` and `print` statements to check if the `score` is less than 60 but greater than or equal to 50, and then print the letter grade C.
9. Write `elif` and `print` statements to check if the `score` is less than 50 but greater than or equal to 45, and then print the letter grade C-.
10. Write `elif` and `print` statements to check if the `score` is less than 45 but greater than or equal to 40, and then print the letter grade D
11. Finally add the following statement to print the letter grade F using the `else` statement;

```
else :
```

```
    print("Grade F")
```

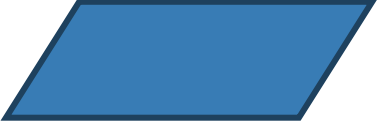
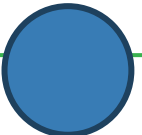
increase / maintain after if or for
decrease to indicate end of block



```
x = 5
if x > 2 :
    print('Bigger than 2')
    print('Still bigger')
print('Done with 2')

for i in range(5) :
    print(i)
    if i > 2 :
        print('Bigger than 2')
    print('Done with i', i)
print('All Done')
```

Flowcharts

	The start or end of the program. There may be more than one way to complete the algorithm and there may be more than one end box.
	A process, that is doing something for example calculating something.
	An input or output , for example: Input num1
 	A decision , YES or NO, or a choice of paths, for example: Is it a weekday?
	When a flowchart will not fit onto a single page we use this shape to show how the sections of the flowchart connect together.

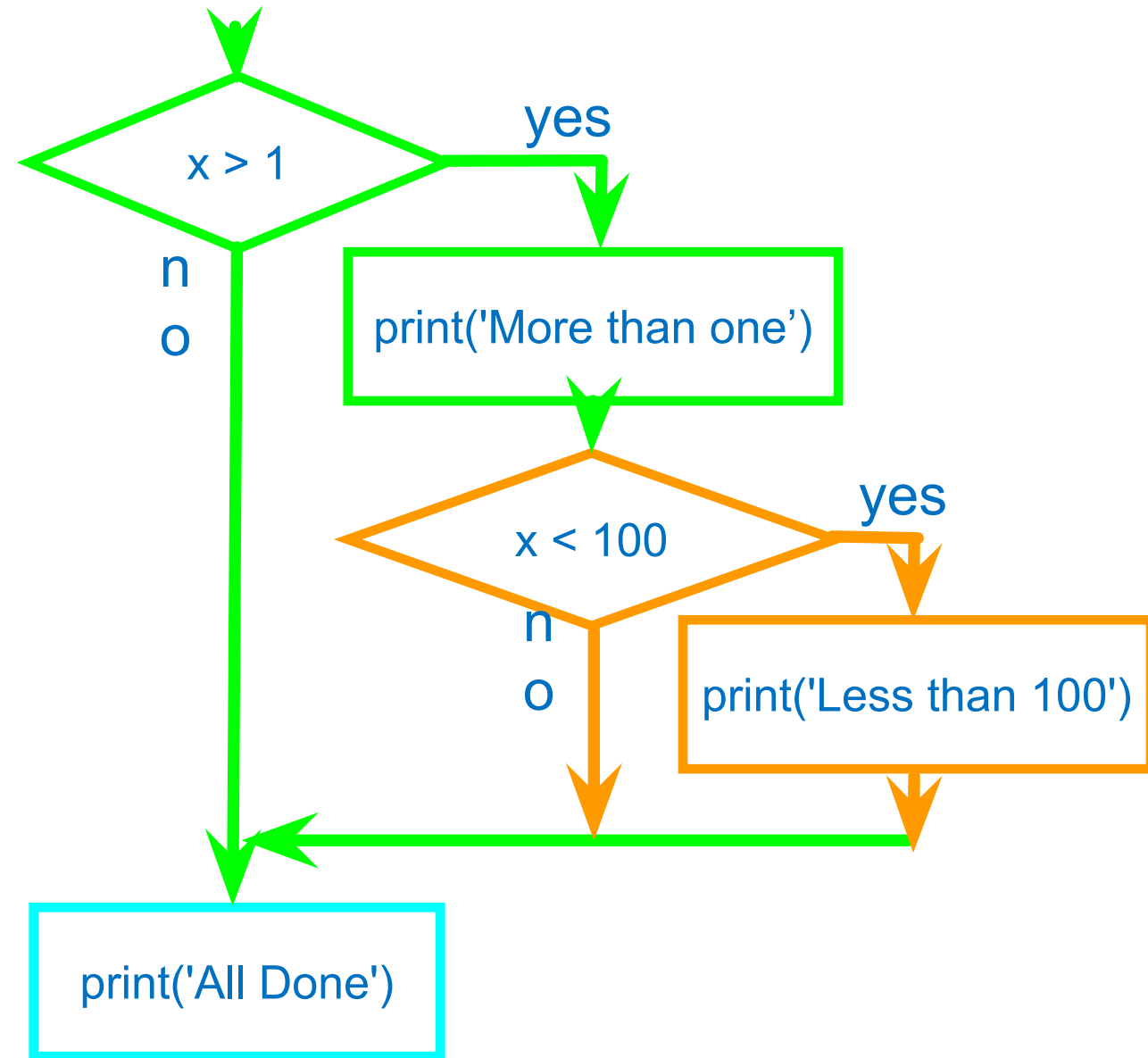
About begin/end Blocks

```
x = 5
if x > 2 :
    print('Bigger than 2')
    print('Still bigger')
print('Done with 2')
```

```
for i in range(5) :
    print(i)
    if i > 2 :
        print('Bigger than 2')
    print('Done with i', i)
print('All Done')
```

Nested Decisions

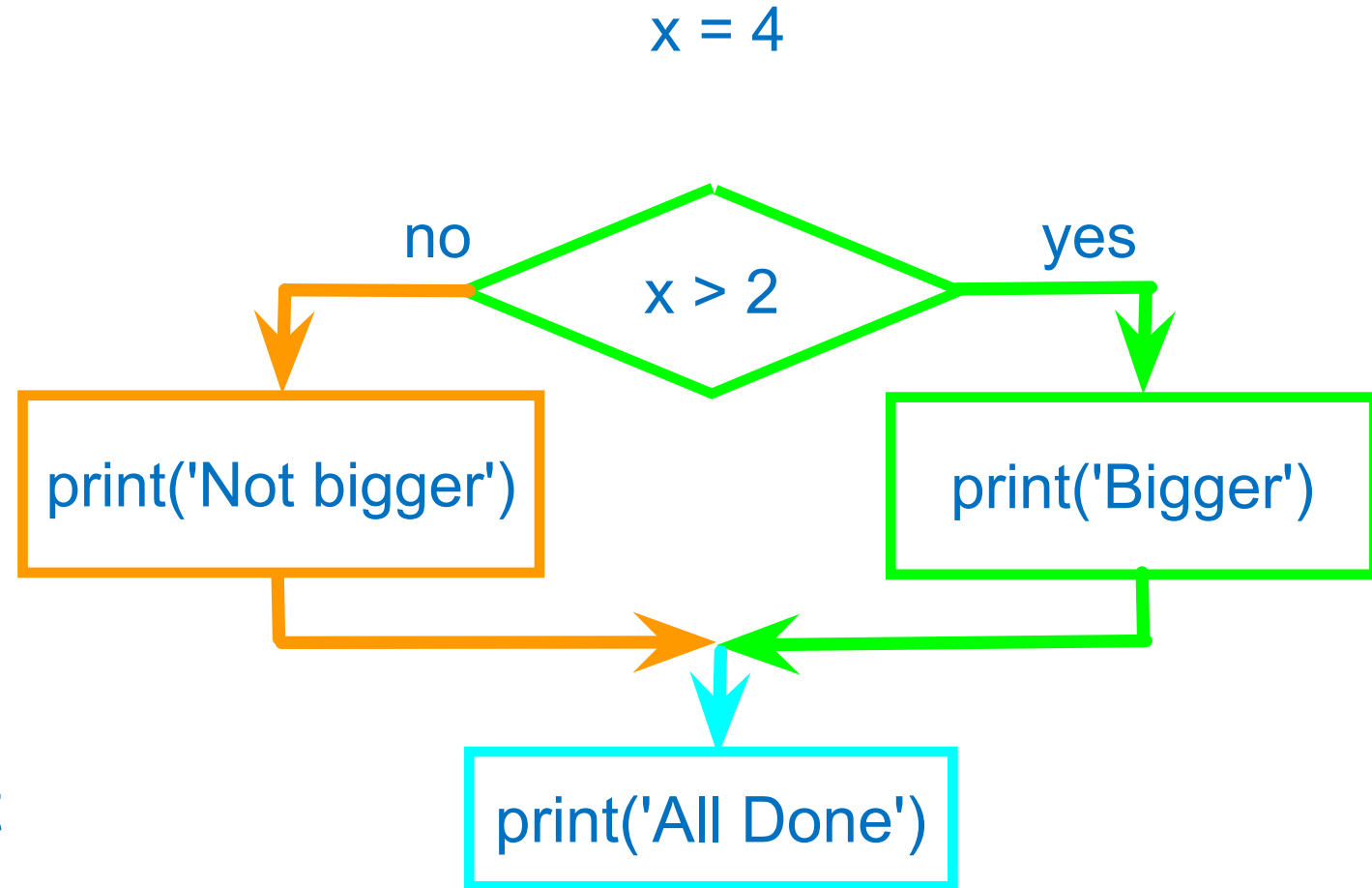
```
x = 42
if x > 1 :
    print('More than one')
    if x < 100 :
        print('Less than 100')
print('All done')
```



Two-way Decisions

Sometimes we want to do one thing if a logical expression is true and something else if the expression is false

It is like a fork in the road - we must choose one or the other path but not both

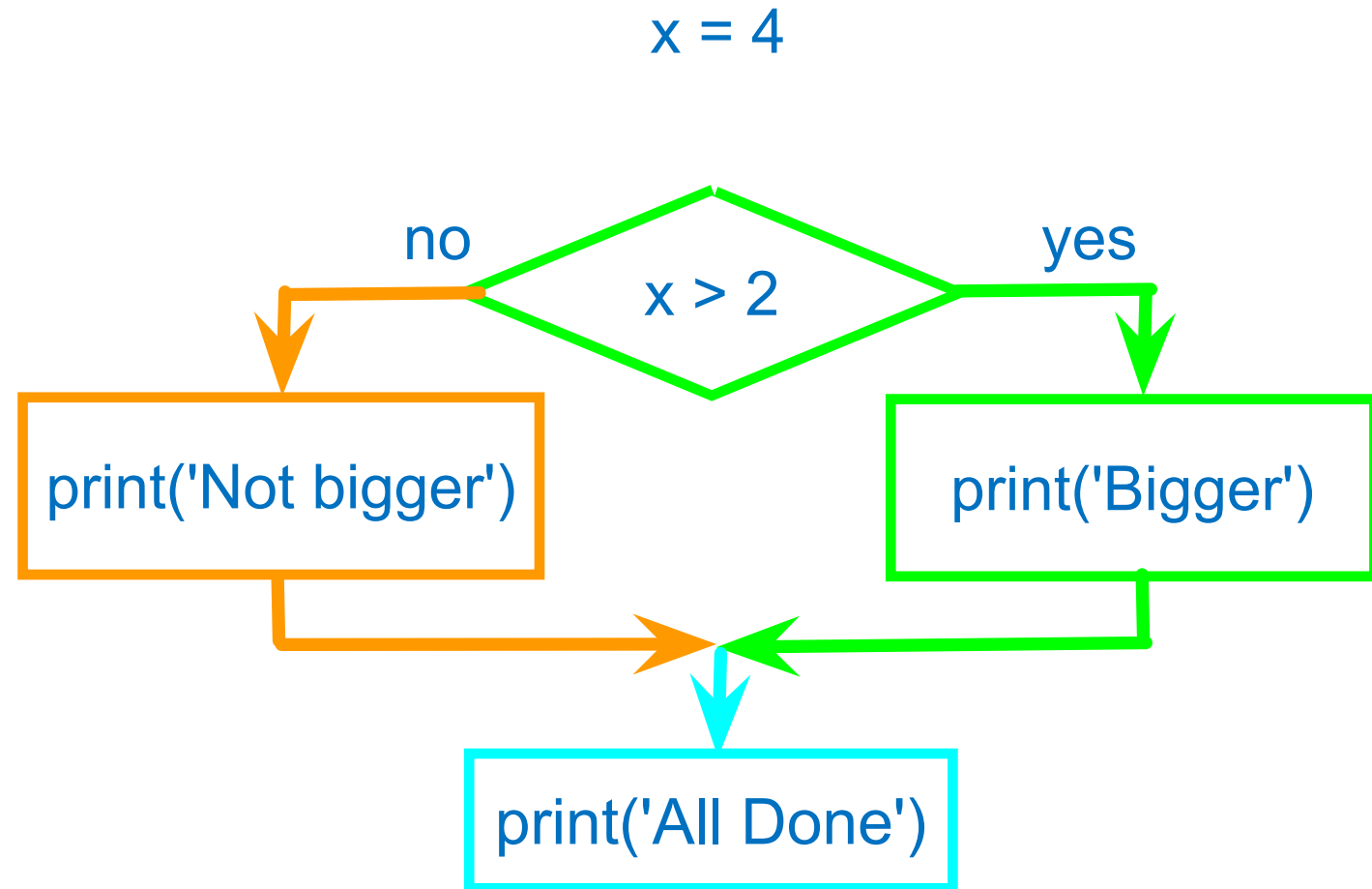


Two-way Decisions with else:

```
x = 4
```

```
if x > 2 :  
    print('Bigger')  
else :  
    print('Smaller')
```

```
print('All done')
```



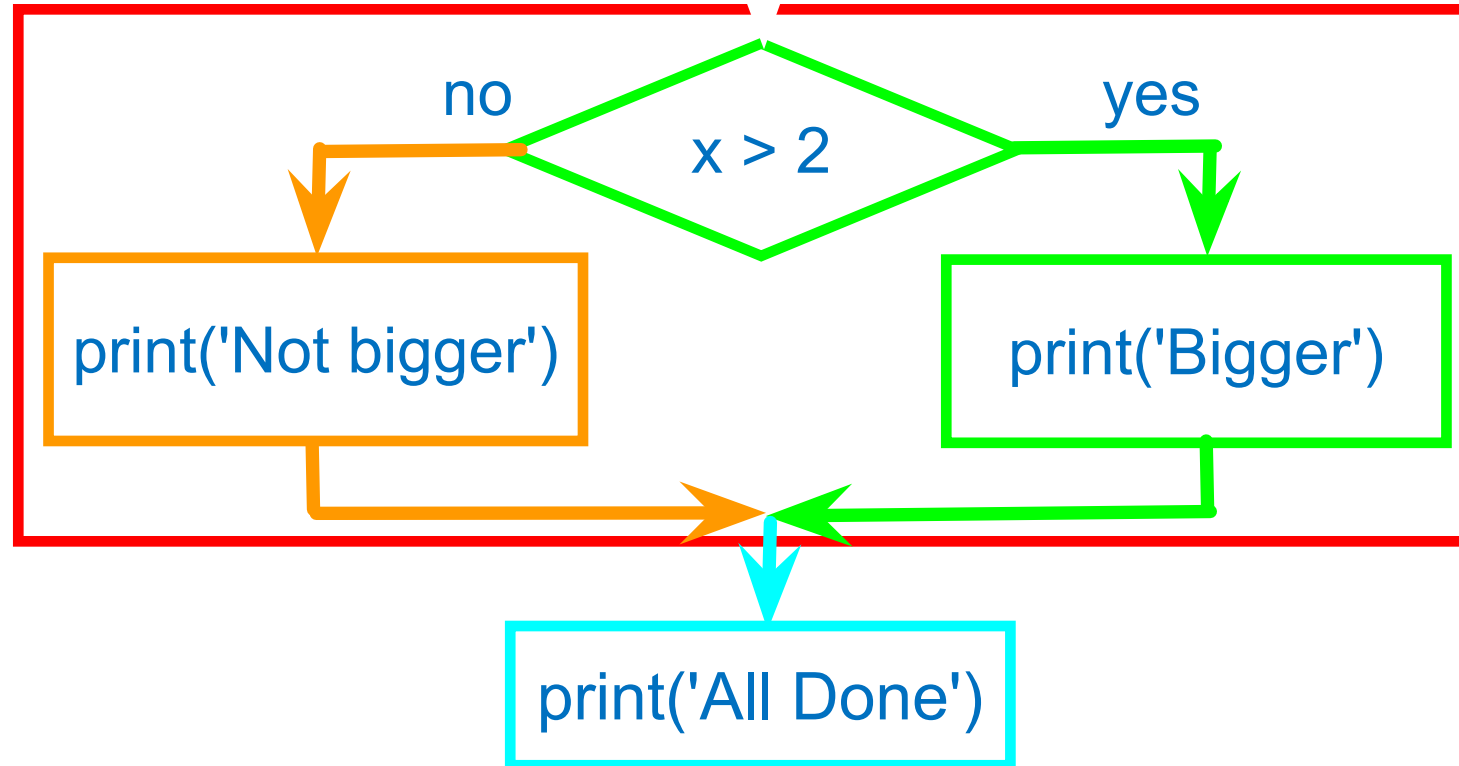
Visualize Blocks

x = 4

```
x = 4
```

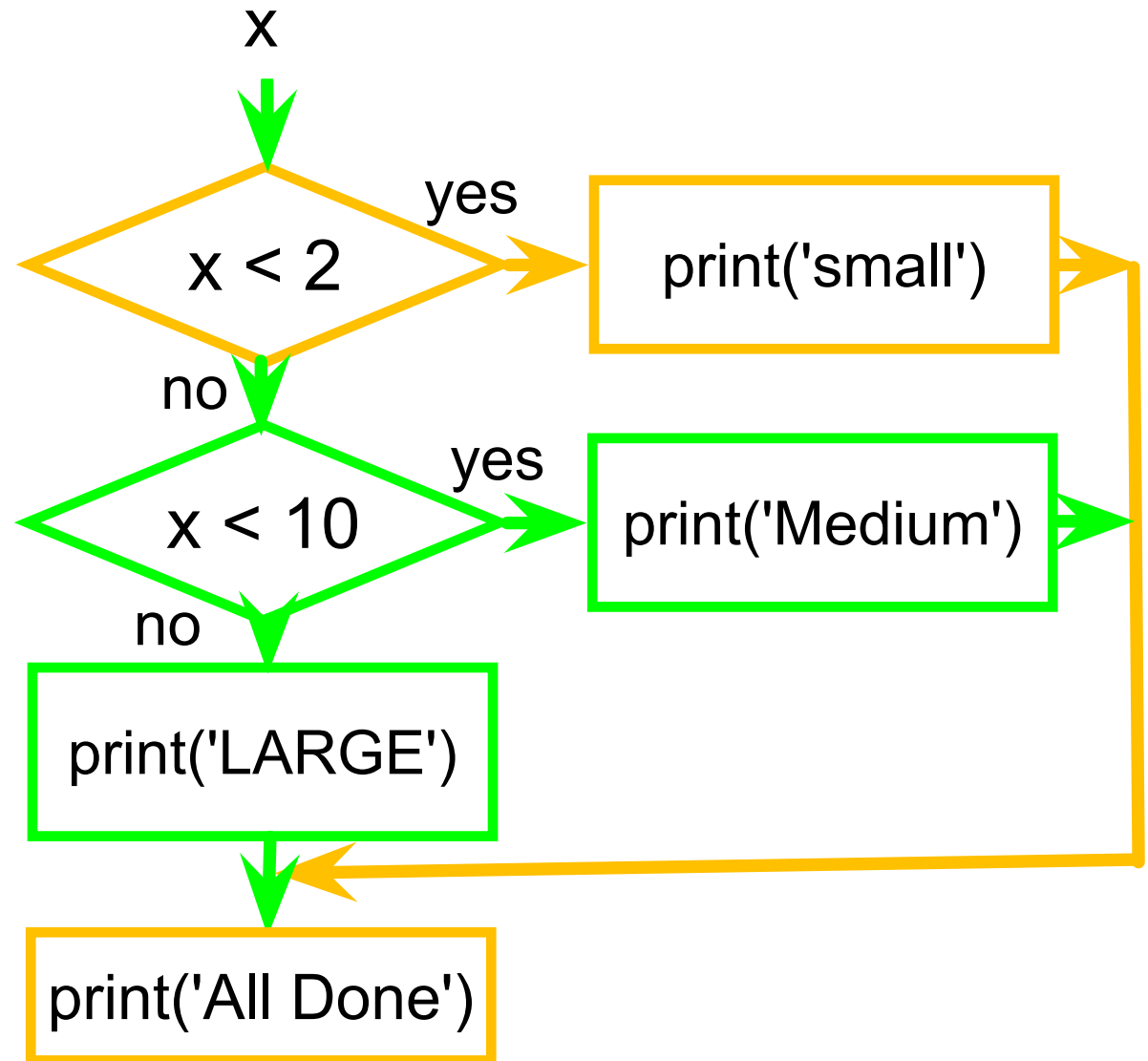
```
if x > 2 :  
    print('Bigger')  
else :  
    print('Smaller')
```

```
print('All done')
```



Multi-way

```
x = int(input("enter valu"))
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



Multi-way

```
# No Else
x = int(input("enter value"))
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')

print('All done')
```

```
x = int(input("enter value"))
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')
elif x < 20 :
    print('Big')
elif x < 40 :
    print('Large')
elif x < 100:
    print('Huge')
else :
    print('Ginormous')
```

Multi-way Puzzles

1. Which statement (s) will never print regardless of the value for x?
2. which statement (s) will never print if x is -2? Why not?

```
x = int(input("enter a value:"))
if x < 2 :
    print('Below 2')
elif x >= 2 :
    print('Two or more')
else :
    print('Something else')
```

```
x = int(input("enter a value:"))
if x < 2 :
    print('Below 2')
elif x < 10 :
    print('Below 100')
elif x < 20 :
    print('Below 20')
else :
    print('Something else')
```

Defensive Programming

- is a programming practice that aims at
 - improving the robustness and reliability of software; and how?
 - by anticipating and planning for potential errors and unexpected conditions.
- The purpose of defensive programming is to ensure that a program continues to function correctly even under unforeseen circumstances.

Defensive Programming

- You can surround a dangerous section of code with **try** and **except** block
 - If the code in the try works, the except is skipped
 - If the code in the try fails however, control jumps to the except section

```
num = int(input("Enter num: "))
try:
    # Try to execute some code that may raise an exception
    result = 10 / num # will raise a ZeroDivisionError if num ==0
    print(result)
except ZeroDivisionError:
    # Handle the exception if it occurs
    print("Error: Division by zero!")
```

Defensive Programming cont'd

- **Input Validation:** validate inputs from users or other systems to ensure they meet the expected format, type, and constraints
 - prevents invalid data from causing errors or security vulnerabilities.

```
age = input("Enter a positive number: ")
try:
    age = int(age)
    if age <= 0:
        raise ValueError("age must be positive.")
    else:
        age = age + 1
        print(f"You are now {age - 1} years old and you will be {age} years old next year!")
except ValueError as e:
    print(f"Invalid input: {e}")
```

Takeaway exercises

1. Write a simple program that accepts a whole number from the keyboard and prints whether the number entered is odd or even
 - Make sure your code controls validity of the input data using try/except
2. Write a simple program that accepts three positive real numbers and displays the average of the numbers,
 - include in your code validity control using try/except

Iterative (Loops)

Outline

- Looping Basics
- The while loop
- Break and Continue
- For Loops, Nested Loops
- Nested Lists & Looping

Looping Basics

- Iteration refers to the process of repeatedly executing a block of code multiple times.
 - It allows you to loop over a sequence of elements, such as lists, tuples, strings, or other iterable objects, and perform some action on each element.
- Looping statements are used to do tasks repeatedly. But, they have to be finite.
- Python provides two types of looping statements to do repetitive tasks and they are **while** loop and **for** loop
 - while loop: are used to iterate over a block of code as long as a certain condition is met.
 - for loop: are used to iterate over a sequence of values. This could be a list, tuple, string, or any other iterable object

The while loop

- syntax

```
while(condition):  
    <statements>
```

- More formally, here is the flow of execution for a while statement:
 - Evaluate the condition, yielding True or False.
 - If the condition is false, exit the while statement and continue execution at the next statement.
 - If the condition is true, execute the body of the loop

while loop

- **Condition-Based:** Continues to iterate as long as the specified condition remains true.
- **Unknown Iterations:** Useful when the number of iterations is not known in advance.
- **Potentially Infinite:** Can result in an infinite loop if the condition never becomes false.
- **Flexibility:** More flexible for situations where the loop's continuation depends on more complex logic or dynamic conditions.
- **Manual Control:** Requires explicit handling of the loop control variable(s) and condition updates within the loop body.

Example

- Count from 1 to 10 and display enclosed between the strings 'start' and 'end'

```
print('start counting')
i = 1 # initialization (done only once)
while(i <= 10): # test condition to enter while body
    print(i) # statement in while body
    i = i + 1 # update statement to control proper iteration
print('stop counting') #control stops looping
```

The break statement

- The break statement is used to exit a loop early. It can be used in both for loops and while loops

```
i = 0
while(i <= 10):
    if(i == 5):
        break
    print(i)
    i = i + 1
print(f"The current value of i is {i}")
```

The continue statement

- The continue statement is used to skip the current iteration of a loop and continue with the next iteration.

```
i = 0
while(i < 10):
    i += 1
    if(i == 5):
        continue
    print(i)
```

Example

```
#program that prints only odd numbers
#between 0 and 10
i = 0
while(i < 10):
    i = i + 1
    if(i % 2 == 0):
        continue
    print(i)
```

for loop

- It works on a sequence of some type like string or items in a collection
- loop in Python has a clear structure and is often used when the number of iterations is known beforehand.

– Syntax

for variable_name **in** sequence:
 statement(s)

for variable_name **in** sequence :
 statement(s)

else:
 statement(s)

for loop

- **Predefined Iterations:** The number of iterations is typically known or can be determined at the start.
- **Sequence-Based:** Iterates directly over items of a sequence or iterable.
- **Concise and Readable:** Provides a straightforward way to loop through elements.
- **Supports Iterators and Generators:** Can work with any object that supports iteration (e.g., files, generators).
- **Range Usage:** Often used with `range()` to iterate a specific number of times

for loop

- an **iterable** is any object capable of returning its members one at a time, allowing it to be looped over in a for loop or used in a variety of other contexts that require sequential access to its elements.
- The concept of iterables is foundational to Python's handling of sequences and collections.

for loop

- Common iterable types
 - **Sequences:** These include lists, tuples, and strings, which are ordered collections of items.
 - **Dictionaries:** Iteration over dictionaries yields keys by default, but values and key-value pairs can also be iterated over
 - **Sets:** These are unordered collections of unique items.
 - **Files:** File objects in Python are iterable, yielding lines of the file

For loop

- The range() function is commonly used in for loop
- The range () function can be called with one, two or three arguments

```
for i in range(5): #starts at 0  
    print(i)
```

```
for i in range(2, 5): #inclusive start, exclusive end  
    print(i)
```

```
for i in range(1, 10, 2): #start, upper limit, offset (jump)  
    print(i)
```

For loop

- Looping in reverse

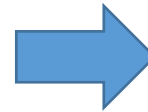
```
for i in range(10, 0, -1): #generates numbers from 10 down to 1
    print(i)
```



10
9
8
7
6
5
4
3
2
1

- Nested loop

```
for i in range(3):
    for j in range(2):
        print(f"i: {i}, j: {j}")
```



i: 0, j: 0
i: 0, j: 1
i: 1, j: 0
i: 1, j: 1
i: 2, j: 0
i: 2, j: 1

Special form of the print function

- Python provides a special form of the print function that prevents it from starting a new line after its arguments are displayed.
- `print(value1, value2, . . ., valuen, end="")`
- Example, the output of the two statements
`print("00", end="")`
`print(3 + 4)`
- is the single line 007 By including `end=""` as the last argument to the first print function, we indicate that an empty string is to be printed after the first argument is displayed instead of starting a new line.
- The output of the next print function starts on the same line where the previous one left off.
- The `end=""` argument is called a **named argument**.

Example nested loop: trace the output and run it

```
##This program prints a table of powers of x

# Initialize constant variables for the max ranges.
NMAX = 4
XMAX = 10

# Print table header.
for n in range(1, NMAX + 1) :
    print("%10d" % n, end="")

print()
for n in range(1, NMAX + 1) :
    print("%10s" % "x ", end="")
print("\n", " ", "-" * 35)

# Print table body.
for x in range(1, XMAX + 1) :
    # Print the x row in the table.
    for n in range(1, NMAX + 1) :
        print("%10.0f" % x ** n, end="")

    print()
```

Takeaway Exercises

1. Why is the newline suppressed (using `end=""`) in the inner loop but not in the outer loop? (previous slide)
2. How would you change the program to display all powers from x^0 to x^5 ?
3. What do the following nested loops display?

```
for i in range(3) :  
    for j in range(1, 4) :  
        print(i + j, end=" ")  
    print()
```

4. Write nested loops that make the following pattern of brackets:

```
[] [] [] []  
[] [] [] []  
[] [] [] []
```

Takeaway Exercises

```
#Q1 What is the output?
for i in range(3):
    for j in range(4):
        print("#", end=" ")
    print()

#Q2. What is the output?
for i in range(4):
    for j in range(6):
        print("$", end=" ")
    print()

#Q3 What is the output?
for i in range(6):
    for j in range(i + 1):
        print("@", end=" ")
    print()
```

```
#Q4 What is the output?
for i in range(4):
    for j in range(6):
        if j % 2 == 0:
            print("#", end=" ")
        else:
            print("&", end=" ")
    print()

#Q5 What is the output?
for i in range(6):
    for j in range(4):
        if i % 2 == j % 2:
            print("@", end=" ")
        else:
            print("&", end=" ")
    print()
```

Check the output

```
for i in range(10):  
    if i % 2 == 0:  
        print(f"{i} is even")  
    else:  
        print(f"{i} is odd")
```

```
for i in range(10):  
    if i % 2 != 0:  
        continue  
    print(i)
```

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

while loop vs for loop

Feature	while Loop	for Loop
Iteration Basis	Iterates based on a condition	Iterates over a sequence or iterable
Typical Usage	When the number of iterations is unknown	When the number of iterations is known
Initialization	Initial condition must be set explicitly	Sequence or iterable is provided
Condition Check	Checked before each iteration	Not explicitly checked, sequence-driven
Loop Control	Must be managed manually within the loop	Handled by the sequence/iterable
Common Functions	Typically involves counters or state variables	Often used with range(), enumerate(), and zip()
Readability	More flexible but can be less readable	More readable for simple sequence iteration
Risk of Infinite Loop	Higher risk if condition management is poor	Low risk if sequence is well-defined

Break and continue

- Break statement is used to end looping statements by checking some condition.
- When a looping statement encounters a **break** statement, it breaks out of the loop and execute the statement that comes after the loop
- Continue statement is used to skip the current execution of a looping statement based on some condition.
- When a looping statement encounters a **continue** statement it skips the current execution jumps to the next iteration

Takeaway exercises

1. Write a simple program that asks for a whole number and displays factors of the number
2. Write a simple program that accepts a number of real numbers decided by the user and displays the average of the numbers entered. Use appropriate input/output messages that convey the purpose
 - Make sure to include error handling code using try/except block

End Chapter 4&6